

DESIGN & ANALYSIS OF ALGORITHMS

BCSE204L | Previous Year Questions Bank

7 Papers

70+ Questions

7 Modules

2022–26

Paper	Key Topics
Undated Paper	Recursion, Knapsack, D&C, KMP, Floyd-Warshall, Ford-Fulkerson, TSP, NP
Winter 2022-23	Karatsuba, Knapsack, Huffman, Matrix Chain, KMP/LPS, Ford-Fulkerson, Graham Scan
April 2023 (G2+TG2)	Recurrence, Fractional Knapsack, Matrix Chain, Rabin-Karp, Bellman-Ford, Convex Hull
May 2024 (A1+TA1)	Matrix Chain, KMP, Floyd-Warshall, Graham Scan, Quicksort, NP, Edmonds-Karp, N-Queens
May 2024 (A2+TA2)	Recurrence, Edit Distance, KMP, Bellman-Ford, Jarvis March, Quicksort, Ford-Fulkerson
April 2025 (A1+TA1)	Karatsuba, Huffman, Held-Karp TSP, Branch & Bound, KMP+Rabin-Karp, Bellman-Ford
April 2026 (A1+TA1)	Fractional Knapsack, Master Theorem, Max Subarray, N-Queens, TSP, Rabin-Karp, Floyd-Warshall

■ MOST REPEATED TOPICS (appear in 5+ papers)

Ford-Fulkerson / Max Flow	7/7 papers
Knapsack (0-1 & Fractional)	7/7 papers
Floyd-Warshall	6/7 papers
KMP String Matching	6/7 papers
Matrix Chain Multiplication	5/7 papers
Convex Hull (Graham/Jarvis)	5/7 papers
Randomized Quicksort	5/7 papers
NP-Completeness / 3-CNF SAT	5/7 papers

■ OFFICIAL SYLLABUS (BCSE204L) — MODULE MAPPING

Module 1	Design Paradigms: Greedy, Divide and Conquer	CO1
Remember	Overview and Importance of algorithms	
Analyze	Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive time complexity	
Apply	Proof of correctness of the algorithm	
Apply	Greedy techniques: Fractional knapsack problem	
Apply	Greedy techniques: Huffman coding	
Apply	Divide and conquer: Maximum subarray	
Apply	Divide and conquer: Karatsuba faster integer multiplication algorithm	
Module 2	Design Paradigms: Dynamic Programming, Backtracking and Branch and Bound	CO2
Apply	Dynamic programming: Assembly line scheduling	
Apply	Dynamic programming: Matrix chain multiplication	
Apply	Dynamic programming: Longest common subsequence	
Apply	Dynamic programming: 0-1 knapsack problem	
Apply	Dynamic programming: TSP	
Apply	Backtracking: N-Queens problem	
Apply	Backtracking: Subset sum problem	
Apply	Backtracking: Graph coloring problem	
Apply	Branch and Bound, LIFO-BB: Job selection problem	
Apply	FIFO-BB: Job selection problem	
Apply	LIFO-BB: 0-1 Knapsack problem	
Apply	FIFO-BB: 0-1 Knapsack problem	
Module 3	String Matching Algorithms	CO3
Apply	Naive string matching algorithm	
Apply	KMP algorithm	
Apply	Rabin-Karp algorithm	
Apply	Suffix trees	
Module 4	Graph Algorithms	CO3
Apply	All pair shortest path: Bellman Ford algorithm	
Apply	Floyd-Warshall algorithm	
Understand	Network Flows: Flow Networks	
Apply	Maximum Flows: Ford-Fulkerson algorithm	
Apply	Edmond-Karp algorithm	

Apply	Push Re-label algorithm, Application of Max Flow to maximum matching problem
-------	--

Module 5	Geometric Algorithms	CO3
Understand	Line Segments — properties, Intersection	
Understand	Sweeping lines	
Apply	Convex Hull: Graham's Scan algorithm	
Apply	Convex Hull: Jarvis March algorithm	

Module 6	Randomized Algorithms	CO4
Apply	Randomized quick sort	
Apply	Hiring problem	
Apply	Finding the global minimum cut	

Module 7	Classes of Complexity and Approximation Algorithms	CO5
Understand	Class P, Class NP, Reducibility and NP-completeness	
Apply	SAT, 3SAT	
Apply	Independent set	
Apply	Clique	
Analyze	Approximation algorithm: Vertex cover	
Analyze	Approximation algorithm: Set cover	
Analyze	Approximation algorithm: Travelling salesman	

Module 1: Algorithmic Analysis & Recurrence Relations [CO1] | 6 Questions

Asymptotic Notation | Recurrence Relations | Recursive Tree | Master Theorem

■ Syllabus: Module 1 (CO1) — Algorithm Analysis, Correctness

Q1

10
M

★
REPEATED

[Undated] [Apr 2026]

Obtain an upper bound for $T(n) = 3T(n/4) + n^2$ using the recursive tree method and verify using Master's theorem.

■ This EXACT question appeared in 2 papers. Master's theorem Case 3 applies here.

Q2

4
M

[Apr 2023]

Solve $T(n) = 3T(n/3) + O(n^{1/2})$ and $T(n) = 4T(n/2) + n^2$ using the Master method.

Q3

4
M

[May 2024 B]

Solve the recurrence $T(n) = 4T(\sqrt{n}) + \log n$ using Master method.

Q4

6
M

[Apr 2023]

Write a non-recursive algorithm to find the sum of first n odd numbers. Using loop invariant property, prove the correctness of the algorithm.

Q5

10
M

[May 2024 A]

Design an algorithm for sum of all elements of a 2D matrix and analyze its time complexity. How to prove correctness using Induction method.

Q6

6
M

[May 2024 B]

Write an iterative algorithm to find sum of $1/i(i-1)$ for $i=2$ to n . Prove your algorithm is correct.

Module 2: Divide and Conquer [CO1] | 5 Questions

Karatsuba Multiplication | Maximum Subarray (D&C;) | Merge Sort | Quick Sort

■ Syllabus: Module 1 (CO1) — Divide & Conquer: Maximum Subarray, Karatsuba

Q1

10
M

★
REPEATED

[May 2024 A]

How does Karatsuba algorithm handle numbers with odd number of digits? Write the algorithm/program and show stepwise execution for binary numbers 10101 and 11. Analyse time complexity.

■ Karatsuba appeared in 2 papers. Know the 3-multiplication trick.

Q2

10
M

★
REPEATED

[Apr 2025]

The Karatsuba algorithm is a divide-and-conquer approach for fast integer multiplication. Break it into stages: problem decomposition, recursive subproblems, and recombination. Derive and analyse its time complexity comparing with standard multiplication. Illustrate for 21×143 .

Q3

10
M

★
REPEATED

[Undated]

Design a Divide and Conquer algorithm to find the Maximum Subarray whose sum is maximum. Consider array $[2, -4, 1, 9, -6, 7, -3]$. Trace the steps with recursive tree. Discuss time complexity.

■ Classic D&C; problem. Know how to split into left/right/cross subarray.

Q4

10
M

★
REPEATED

[Apr 2026]

Design the pseudocode for Divide and Conquer Maximum Subarray. Apply algorithm on $A = [-2, 5, -1, 3, -4, 7, -2, 1]$. Show the left sum, right sum, and cross sum for each recursive division. State the starting and ending indices of the maximum subarray and its sum.

Q5

10
M

[May 2024 B]

You are given two DNA sequences DNA1 and DNA2. Find the minimum number of single-character edits (insertions, deletions, or substitutions) to transform DNA1 into DNA2. Given DNA1='AGTACG' and DNA2='GTCAGT', minimum edits = 4. Develop a DP algorithm for minimum edit distance.

■ This is Edit Distance — a DP topic (Module 2 CO2).

Module 3: Greedy Algorithms [CO2] | 9 Questions

Fractional Knapsack | Activity Selection | Huffman Coding | Job Scheduling

■ Syllabus: Module 1 (CO1) — Fractional Knapsack, Huffman Coding

Q1

10
M

★
REPEATED

[Undated]

Solve the Fractional Knapsack problem using Greedy strategy. Knapsack capacity is 35. Weights = [10, 20, 12], profits = [60, 100, 120]. Find optimal items to include.

■ Sort by profit/weight ratio. This exact instance appeared multiple times.

Q2

10
M

★
REPEATED

[Apr 2023]

State the fractional Knapsack problem. Use greedy method for $n=7$, capacity $W=23$. Profits = [29,17,20,19,6,28,42]. Weights = [7,5,6,3,9,2,5]. Show all steps.

Q3

5
M

★
REPEATED

[Apr 2026]

Design an algorithm for the Fractional Knapsack problem. Profits=[5,10,15,7,8,9], Weights=[1,3,5,4,1,3], Capacity $W=15$. Show profit/weight ratio calculations. Why is Fractional Knapsack solved using Greedy while 0/1 Knapsack cannot be?

Q4

5
M

★
REPEATED

[Winter 2022-23]

Validate: 'Greedy approach does not ensure optimal solution for 0-1 Knapsack' using two examples — capacity 8 (weights [3,4,6,5], values [20,30,10,40]) and capacity 20 (weights [15,10,10], values [9,5,5]).

Q5

10
M

★
REPEATED

[Apr 2025]

Construct the optimal Huffman code for characters a-h with frequencies from first 8 prime numbers [2,3,5,7,11,13,17,19]. Derive time and space complexity. Show Huffman coding takes less space than fixed size code.

■ Huffman appeared in 3 papers. Build tree bottom-up, always merge two smallest.

Q6

5
M

[Winter 2022-23]

Two encoding schemes E1 and E2 encode characters R,G,B,Y as (0,011,111,11) and (0,11,101,100). String 'RGRBYRR' is encoded using both. Which is Huffman encoding? Justify.

Q7

10
M★
REPEATED

[May 2024 B] [Apr 2026]

Find a greedy approach to attend maximum sessions without conflicts. Sessions: A(9:00-10:00), B(9:30-11:30), C(11:00-12:00), D(12:30-14:00), E(13:00-14:30), F(14:30-15:00).

■ *Activity Selection — sort by end time, pick non-overlapping greedily.*

Q8

10
M

[May 2024 A]

Define Greedy Knapsack. $n=7$, $M=15$, profits=[10,5,15,7,6,18,3], weights=[2,3,5,7,1,4,1]. Find optimal solution.

Q9

10
M

[May 2024 A] [May 2024 B]

Solve tasks with deadlines and profits to produce maximum profit. Jobs J1-J5, Deadlines=[2,2,1,3,4], Profits=[20,60,40,100,80].

Module 4: Dynamic Programming [CO2] | 7 Questions

Matrix Chain Multiplication | TSP (Held-Karp) | 0-1 Knapsack DP
| LCS

■ Syllabus: Module 2 (CO2) — Matrix Chain, LCS, 0-1 Knapsack, TSP

Q1

10
M

★
REPEATED

[Undated]

Design a dynamic programming algorithm for parenthesization of matrix chain requiring minimum scalar multiplications. Consider chains A(2x3) B(3x2) C(2x4) D(4x3). Illustrate the algorithm.

■ Matrix Chain is one of the most classic DP problems. Always in exams.

Q2

10
M

★
REPEATED

[May 2024 A]

Outline important factors of Dynamic Programming and perform Matrix Chain Multiplication for {4,10,3,12,20,7}. Compute $M[i,j]$, $0 \leq i, j \leq 5$.

Q3

10
M

★
REPEATED

[Apr 2023]

Given a sequence of matrices <8,5,10,20,5>. Using Dynamic Programming find the minimum number of scalar multiplications needed and also write the optimal multiplication order.

Q4

10
M

★
REPEATED

[Winter 2022-23]

Given DP pseudocode for matrix chain multiplication — explain reasons for assigning initial/final values for i loop, k loop, and j variable. Fill the M table for 5 matrices [4,10,3,12,20,7].

Q5

10
M

★
REPEATED

[Apr 2025]

A logistics company wants to optimize delivery route for a salesperson visiting 4 cities exactly once. Using Held-Karp Algorithm (Dynamic Programming), compute the minimum travel cost and show step-by-step calculations. Compare DP vs brute-force complexity.

■ Held-Karp TSP appeared in 3 papers. Know $g(S,i) = \min \text{ over } k \text{ in } S \text{ of } [g(S-\{i\},k) + \text{cost}(k,i)]$.

Q6

10
M

★
REPEATED

[Apr 2026]

Solve the Travelling Salesman Problem using Dynamic Programming. Cost Matrix (cities A,B,C,D): A-B=20, A-C=42, A-D=35, B-C=30, B-D=34, C-D=12. Starting city: A. Show all calculations.

Q7

10
M

★
REPEATED

[Undated]

Design a polynomial time approximation algorithm for the Travelling Salesman Problem. Illustrate for the given graph (starting vertex 1). Prove it is a 2-approximation algorithm.

■ Also maps to Module 7 (CO5) — TSP Approximation.

Module 5: Backtracking [CO2] | 4 Questions

[N-Queens Problem](#) | [Subset Sum Problem](#) | [0-1 Knapsack via Backtracking](#)

■ Syllabus: Module 2 (CO2) — N-Queens, Subset Sum, Graph Coloring

Q1

10
M

★
REPEATED

[May 2024 A]

Explain N-Queens algorithm with 4x4 chessboard, numbering rows and columns 1 through 4, in a State Space Tree using Backtracking (step-by-step). Derive Time Complexity.

■ N-Queens is a standard backtracking problem. Draw the state space tree.

Q2

10
M

★
REPEATED

[Apr 2026]

Consider the N-Queens problem for N=6. Construct the state-space tree up to level 3. Indicate promising and non-promising nodes. Show complete backtracking process for one valid solution. Analyse worst-case time complexity.

Q3

10
M

★
REPEATED

[Winter 2022-23]

Write pseudocode for implementing backtracking to find the subset of elements from a given set whose sum adds up to a given target. Find the first two subsets from [10,20,50,60,80] that sum to 80.

■ Subset sum backtracking — know when to prune (current sum > target).

Q4

10
M

★
REPEATED

[May 2024 B]

Define the Subset Sum problem. Solve it for the set {40,30,50,10,20} with target sum 60. Use backtracking approach.

Module 6: Branch and Bound [CO2] | 3 Questions

0-1 Knapsack FIFO B&B; | Job Scheduling B&B; | State Space Tree

■ Syllabus: Module 2 (CO2) — LIFO-BB & FIFO-BB: Job selection, 0-1 Knapsack

Q1

10
M

★
REPEATED

[Undated]

Solve the Knapsack problem using fixed length FIFO branch and bound strategy. 4 items, capacity 15, Profits={10,12,20,18}, Weights={2,4,5,9}. Draw annotated state space tree with cost and bound values.

■ Key formula: $upper_bound = current_profit + remaining_capacity \times (next\ item\ p/w\ ratio)$.

Q2

10
M

[Apr 2023]

Given $n=4$ jobs with penalties $(p_1, p_2, p_3, p_4) = (5, 10, 6, 4)$, deadlines $(d_1, d_2, d_3, d_4) = (1, 2, 2, 1)$, execution times $(t_1, t_2, t_3, t_4) = (1, 2, 1, 2)$. Find the actual cost and feasible jobs done using branch and bound.

Q3

10
M

★
REPEATED

[Apr 2025]

Apply the FIFO Branch and Bound approach to solve the 0-1 Knapsack Problem. Packages: weight=[2,3,4,5], value=[40,50,70,80]. Contrast Backtracking and Branch & Bound paradigms in terms of search strategy, pruning mechanism, efficiency, and applicability.

Module 7: Graph Algorithms — Shortest Paths & Network Flow [CO3] | 12 Questions

Floyd-Warshall | Bellman-Ford | Ford-Fulkerson |
Edmonds-Karp | Push-Relabel

■ Syllabus: Module 4 (CO3) — Bellman-Ford,
Floyd-Warshall, Ford-Fulkerson, Edmonds-Karp,
Push-Relabel

Q1

10
M

★
REPEATED

[Undated]

Find all pairs shortest path for the following weighted digraph using Floyd-Warshall's Algorithm. Nodes: A,B,C,D,E with given weights. Analyse time complexity. Illustrate the tracing of the algorithm.

■ Floyd-Warshall appeared in 6/7 papers! Know $D^k[i][j] = \min(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j])$.

Q2

10
M

★
REPEATED

[May 2024 A]

Perform step-by-step process of finding shortest path distance between every pair of vertices using Floyd-Warshall algorithm (vertices 1,2,3,4 with given edge weights).

Q3

10
M

★
REPEATED

[Apr 2026]

Find all-pairs shortest path using Floyd-Warshall's Algorithm. Edges: B-A:10, A-C:12, C-B:20, C-D:32, A-D:60, E-A:7. Analyse time complexity. Detect whether a negative cycle exists using diagonal entries.

Q4

10
M

★
REPEATED

[Apr 2023]

Elaborate the procedure to find the shortest path from source to all other vertices using Bellman-Ford algorithm. Vertices A,B,C,D,E. Discuss how dynamic programming strategy is applied. State how Bellman-Ford detects negative weight cycles.

■ Bellman-Ford appeared in 4 papers. Runs $(V-1)$ iterations. If relaxation still possible in V -th iteration, negative cycle exists.

Q5

10
M

★
REPEATED

[May 2024 B]

Given directed weighted graph $G(V,E)$ with potentially negative weight edges, check whether Bellman-Ford can find shortest path. Determine possibility of negative cycles. Graph vertices 0,1,2,3.

Q6

10
M

★
REPEATED

[May 2024 A] [May 2024 B]

Solve Bellman-Ford single source shortest path. Starting vertex '1', destination vertex '7'. Graph has 7 vertices with various weights including negatives. Analyse complexity.

Q7

10
M★
REPEATED*[Undated]*

Apply Ford-Fulkerson's algorithm on the following network flow graph to find the maximum flow from start vertex 'A' to sink vertex 'G'. In each iteration, show the graph traversal, the flow value to be augmented, and the residual flow network after augmenting.

■ *Ford-Fulkerson appeared in ALL 7 papers. This is the #1 most tested topic!*

Q8

10
M★
REPEATED*[Winter 2022-23]*

Apply Ford-Fulkerson algorithm. Justify the need for adding flow along reverse edges in the residual graph. Given 3 networks with augmented paths (0-1-3), (0-1-2-3), (0-1-2-3) respectively. Compute maxflow.

Q9

10
M★
REPEATED*[Apr 2023] [May 2024 B] [Apr 2025] [Apr 2026]*

Apply Ford-Fulkerson — max flow from source 's' to sink 't'. Network with nodes s,a,b,c,d,e,t and various capacities. Show each augmenting path and residual graph.

Q10

10
M★
REPEATED*[May 2024 A]*

Write Edmonds-Karp's algorithm, compute the maximum flow in network N (nodes S,A,B,C,D,T with given capacities: S-A:15, S-B:12, A-C:10, B-C:5, B-D:6, etc.) and analyse its complexity.

■ *Edmonds-Karp is Ford-Fulkerson with BFS for augmenting path. $O(VE^2)$.*

Q11

10
M*[Winter 2022-23]*

Apply Push-Relabel algorithm to find the maxflow. Source=0, sink=4. Graph edges: 0-1:12, 0-2:15, 1-2:7, 1-3:8, 2-4:6, 3-4:5.

Q12

10
M★
REPEATED*[Apr 2025]*

A logistics company wants to maximize shipment flow using Ford-Fulkerson from source s to sink t through cities u,v,w,x,y. First step: route with longest augmenting path. Second step: route with highest residual capacity. Show original graph, augmenting paths, flow network, and residual graphs.

Module 8: String Matching Algorithms [CO3] | 9 Questions

KMP Algorithm (LPS / Failure Function) | Rabin-Karp (Rolling Hash) | Naive String Matching

■ Syllabus: Module 3 (CO3) — Naive, KMP, Rabin-Karp, Suffix Trees

Q1

10
M

★
REPEATED

[Undated]

Apply an algorithm that constructs the Longest Prefix Suffix (LPS) table for pattern 'ABABAD' and text 'ABCABABABAD'. Return the position where the pattern is found. Analyze time complexity.

■ KMP appeared in 6/7 papers. Always build LPS table first!

Q2

10
M

★
REPEATED

[Winter 2022-23]

Construct the LPS table for pattern '121231213'. What is the length of the longest proper prefix and suffix? If characters from indexes 0 to 6 match but 7th does not, which character should next comparison start from?

Q3

5
M

★
REPEATED

[May 2024 A]

Interpret steps of KMP_next array and the KMP algorithm. Compute the KMP_next array of Text=ABCCDDAEFG and pattern=CDD.

Q4

10
M

★
REPEATED

[May 2024 B]

Write an algorithm to report the index of the first occurrence (if any) of P in T from the pi-table of PT where PT is the concatenation of P and T. Trace with a positive example.

Q5

10
M

★
REPEATED

[Apr 2025]

Apply KMP and Rabin-Karp for pattern P='ACGTACGT' within DNA sequence T='GCACGTACGTACGTACGTA'. For Rabin-Karp: A=1, C=2, G=3, T=4, base=4, mod=101. Compare efficiency.

Q6

5
M

[May 2024 A]

Classify between KMP and Rabin-Karp algorithm — strength and weakness of complexity.

Q7

10
M

★
REPEATED

[Apr 2023]

For string matching with hash function $\text{Pattern} \bmod q$ where $q=13$, how many spurious hits does Rabin-Karp encounter in Text $T=42637135421$ and $P=54$? Find number of shifts.

■ Rabin-Karp: $H_{\text{new}} = (d \cdot (H_{\text{old}} - T[s] \cdot h) + T[s+m]) \bmod q$.

Q8**10
M**★
REPEATED*[May 2024 B]*

Write pseudocode for Rabin-Karp algorithm. Trace for text $T = \text{adbadbacfegba}$ and pattern $P = \text{ba}$. Hash code: $a=1, b=2, \dots, d=8, q=13$. Demonstrate rolling hashing function.

Q9**10
M**★
REPEATED*[Apr 2026]*

Rabin-Karp — Text $T = \text{'14159265358979323846'}$ (first 20 digits of π), Pattern $P = \text{'59265'}$. Base $d=10$, Prime $q=13$. Compute initial hash values, show rolling hash for next 3 shifts, identify spurious hits.

Module 9: Geometric Algorithms [CO3] | 7 Questions

Convex Hull — Graham's Scan $O(n \log n)$ | Convex Hull — Jarvis March $O(nh)$ | Line Segment Intersection

■ Syllabus: Module 5 (CO3) — Line Segments, Sweeping Lines, Graham's Scan, Jarvis March

Q1

10
M

★
REPEATED

[Undated] [Apr 2026]

Solve the Convex Hull using Graham's Scan $O(n \log n)$ for points $Q=\{P1(10,3), P2(6,1), P3(8,4), P4(2,2), P5(3,5), P6(1,6), P7(7,3), P8(6,7)\}$. Elaborate the step-by-step process.

■ *Graham's Scan appeared in 5/7 papers! Same point set repeated. Sort by polar angle from lowest point.*

Q2

10
M

★
REPEATED

[May 2024 A]

Relate Graham's Scan and Jarvis March Algorithm steps and how to choose the best practices for computational Convex Hull. Illustrate with an example.

Q3

5
M

★
REPEATED

[Apr 2025]

Apply Graham's Scan algorithm for point set $P=\{(1,2),(3,5),(4,4),(6,2),(7,6),(8,3),(2,7),(5,1)\}$. Illustrate step-by-step execution showing intermediate hull formations. Compare computational efficiency of Graham's Scan and Jarvis' March.

Q4

10
M

★
REPEATED

[May 2024 B]

Write the pseudocode for Jarvis-march algorithm and trace it for the given points $p1...p7$ provided in the diagram.

■ *Jarvis March: $O(nh)$ where h = number of hull points. Repeatedly find most counterclockwise point.*

Q5

10
M

★
REPEATED

[Apr 2023]

Apply Graham's Scan to find the convex hull of points $(2,1)(3,3)(5,4)(-2,2)(-3,1)(-4,3)(-1,-2)(-3,-4)(2,-3)(4,-4)$ in counter clockwise direction.

Q6

10
M

[Undated]

Design an algorithm for checking whether two given line segments intersect. Trace and find if $p1q1$ and $p2q2$ intersect for 4 pairs including: (i) $p1(2,10)$ $q1(7,2)$ $p2(4,15)$ $q2(13,3)$.

■ *Use cross product to determine orientation. 4 cases: general and collinear.*

Q7

5
M

[Winter 2022-23]

From the following set of line segments, find all the pairs that intersect using a more efficient method than Brute Force. Segments: $\{(2,8),(6,9)\}$, $\{(5,7),(10,7)\}$, $\{(3,5),(8,4)\}$, $\{(7,6),(12,12)\}$, etc.

Module 10: Randomized Algorithms & NP-Completeness [CO4/CO5] | 9 Questions

Randomized Quicksort | Hiring Problem | NP Classes | 3-CNF SAT | Clique | Vertex Cover

■ Syllabus: Module 6 (CO4) — Randomized Quicksort, Hiring Problem | Module 7 (CO5) — P/NP, SAT, Clique, Approximation

Q1

10
M

★
REPEATED

[Undated] [Apr 2026]

Apply Randomized Quicksort algorithm by selecting the random pivot to trace sorting steps. Elements: [99,34,46,22,60,23,15,14,33,20]. Discuss how average time complexity of randomized quicksort is better than worst-case of deterministic quicksort.

■ Same array [99,34,46,22,60,23,15,14,33,20] appeared in 3 papers!

Q2

10
M

★
REPEATED

[May 2024 A]

Narrate how Randomized Quick Sort algorithm differs from Normal Quick Sort. Write an algorithm and solve the randomized quick sort algorithm with an example.

Q3

10
M

★
REPEATED

[Apr 2025]

Apply Randomized Quicksort on [9,4,7,2,6,3,8]. Show step-by-step execution including selection of random pivots, partitioning, and recursive calls. Analyse expected time complexity.

Q4

10
M

[Apr 2023] [Winter 2022-23]

Design a randomized algorithm for the hiring problem. Consider cost to interview as c_i per candidate and cost to hire as c_h . Analyse performance and prove that algorithm Hire-Assistant has a hiring cost of $O(c_h \ln n)$.

Q5

10
M

★
REPEATED

[Apr 2025] [May 2024 A]

Provide a summarized explanation of the complexity classes P, NP, NP-Hard, and NP-Complete, highlighting their role in assessing the hardness of real-world computational problems. Demonstrate the NP-completeness of the 3-SAT problem by formally reducing SAT to 3-SAT.

■ NP-completeness appeared in 5 papers. Know the standard reductions: SAT $\rightarrow$ 3-SAT $\rightarrow$ Clique.

Q6

10
M

★
REPEATED

[Undated]

Prove that 3-CNF (Conjunctive Normal Form) SAT is NP complete. Use the satisfiability formula: $\phi = (x_4 \text{ AND } ((x_1 \rightarrow x_2) \text{ AND } (x_3 \leftarrow x_1)))$. Illustrate the three steps for conversion of ϕ to ϕ_i .

Q7

10
M★
REPEATED*[Apr 2023] [Winter 2022-23]*

Prove clique decision problem is NP-complete with the help of 3-CNF SAT: $(a \vee \bar{b} \vee \bar{c}) \wedge (\bar{a} \vee b \vee c) \wedge (a \vee b \vee c)$.

Q8

10
M★
REPEATED*[Apr 2025] [Apr 2026]*

Prove that the Clique Decision Problem is NP-Complete using 3-CNF SAT: $(a \vee \bar{b} \vee c) \wedge (\bar{a} \vee b \vee c) \wedge (a \vee b \vee c)$. Provide summarized explanation of Vertex Cover Problem, its NP-Complete proof, and 2-approximation algorithm.

Q9

10
M*[May 2024 A]*

Brief the various complexity classes and illustrate the conversion of SAT problem to 3-CNF-SAT problem.

■ TOP 10 PREDICTED QUESTIONS — EXAM 2026

Based on April 2026 format: Answer ALL 10 Questions × 10 Marks = 100 Marks. Predictions based on frequency analysis of 7 past papers.

#1

Ford-Fulkerson / Max Flow

Appeared in EVERY single paper. Guaranteed to come.

Apply Ford-Fulkerson's algorithm on the given network flow graph to find the maximum flow from source 's' to sink 't'. In each iteration, show the graph traversal performed to find the augmenting path, the flow value to be augmented, and the residual flow network obtained after augmenting the flow.

#2

Knapsack (Fractional / 0-1)

Appeared in EVERY paper — either Greedy (Fractional) or DP/Branch&Bound; (0-1).

Design a Fractional Knapsack algorithm using Greedy approach. Given profits and weights, show profit/weight ratio calculations and item selection order to find maximum profit for capacity W . Why is Fractional Knapsack solved using Greedy while 0/1 Knapsack cannot be?

#3

6/7

Floyd-Warshall (All-Pairs Shortest Path)

One of the most consistently tested topics. Know the D -matrix iterations.

Find all-pairs shortest path for the given weighted digraph using Floyd-Warshall's Algorithm. Perform D_0 through D_n matrix iterations step by step. Analyse time complexity $O(n^3)$. Detect whether a negative cycle exists using diagonal entries.

#4

6/7

KMP String Matching + LPS Table

LPS table construction and KMP trace are always tested together.

Apply KMP algorithm — construct the Longest Prefix Suffix (LPS / failure function) table for given pattern. Trace pattern matching on given text. State all positions where pattern is found. Compare time complexity $O(n+m)$ with Naive $O(nm)$ algorithm.

#5

5/7

Randomized Quicksort

Same element array [99,34,46,22,60,23,15,14,33,20] appeared in 3 papers!

Apply Randomized Quicksort algorithm by selecting random pivot to trace the steps to sort [99,34,46,22,60,23,15,14,33,20] in detail. Discuss how average time complexity $O(n \log n)$ of randomized quicksort is better than worst-case $O(n^2)$ of deterministic quicksort.

#6

5/7

Convex Hull (Graham's Scan)

Graham's Scan with same point set $P_1(10,3) \dots P_8(6,7)$ appeared in 2+ papers.

Consider a set of points P in a XY plane. Construct a smallest polygon enclosing all the points. Design an $O(n \log n)$ time algorithm and trace it using points $P_1(10,3)$, $P_2(6,1)$, $P_3(8,4)$, $P_4(2,2)$, $P_5(3,5)$, $P_6(1,6)$, $P_7(7,3)$, $P_8(6,7)$. Elaborate the step-by-step process.

#7

5/7

NP-Completeness / 3-CNF SAT / Clique*Either prove NP-completeness or explain P/NP/NP-Hard/NP-Complete classes.*

Provide a summarized explanation of the complexity classes P , NP , NP -Hard, and NP -Complete. Additionally demonstrate the NP -completeness of 3-SAT / Clique Decision Problem by formally performing the appropriate reduction.

#8

4/7

Recurrence Relations + Master Theorem *$T(n)=3T(n/4)+n^2$ appeared in 2 papers. Master Theorem Case 3.*

Obtain an upper bound for $T(n) = 3T(n/4) + n^2$ using the recursive tree method. Draw the recursion tree, compute the cost at each level, and verify the result using Master's theorem.

#9

4/7

Divide & Conquer Max Subarray*Appeared in Apr 2026 format. Know how to compute left/right/cross subarray sums.*

Design the pseudocode for Divide and Conquer Maximum Subarray. Apply the algorithm to find the contiguous subarray with the largest sum in $A=[-2,5,-1,3,-4,7,-2,1]$. Show left sum, right sum, and cross sum for each recursive division.

#1
0

4/7

TSP — Held-Karp DP Algorithm*Growing trend — DP-based TSP is becoming the standard format.*

Solve the Travelling Salesman Problem for given 4-city distance matrix using Held-Karp (Dynamic Programming) algorithm. Show all $g(S,i)$ calculations step by step, determine the minimum cost tour, and trace back the optimal path.

■ QUICK REFERENCE — ALGORITHMS CHEAT SHEET

Algorithm	Complexity	Key Formula / Idea
Floyd-Warshall	$O(n^3)$	$D^k[i][j] = \min(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j])$
Bellman-Ford	$O(VE)$	Relax all edges $(V-1)$ times. V -th relaxation = negative cycle.
Ford-Fulkerson	$O(E \times \text{max_flow})$	BFS/DFS to find augmenting path, update residual graph, repeat.
Edmonds-Karp	$O(VE^2)$	Ford-Fulkerson with BFS only. Guaranteed polynomial.
KMP	$O(n+m)$	Build LPS failure function in $O(m)$, then scan text in $O(n)$.
Rabin-Karp	$O(n+m)$ avg	Rolling hash: $H_{\text{new}} = (d \cdot (H_{\text{old}} - T[s] \cdot h) + T[s+m]) \bmod q$.
Graham's Scan	$O(n \log n)$	Sort by polar angle. Use stack, pop if right turn (CW).
Jarvis March	$O(nh)$	Repeat: find most CCW point from current. Stop when back at start.
Matrix Chain DP	$O(n^3)$	$m[i][j] = \min \text{ over } k \text{ of } m[i][k] + m[k+1][j] + p[i-1] \cdot p[k] \cdot p[j]$.
Held-Karp TSP	$O(2^n \times n^2)$	$g(S, i) = \min_{k \in S - \{i\}} [g(S - \{i\}, k) + \text{cost}(k, i)]$.
Fractional Knapsack	$O(n \log n)$	Sort by profit/weight ratio. Take greedily. Fraction of last item OK.
Karatsuba	$O(n^{1.585})$	3 multiplications instead of 4: $ac, bd, (a+b)(c+d) - ac - bd$.
Randomized Quicksort	$O(n \log n)$ avg	Pick random pivot. Expected $T(n) = O(n \log n)$ regardless of input.
Master Theorem	3 Cases	Case 1: $f = O(n^{\log_b(a)-\epsilon}) \rightarrow \Theta(n^{\log_b(a)})$. Case 2: $\Theta(\dots \log n)$. Case 3: $\Theta(f(n))$.

All the best for your exam! — BCSE204L DAA PYQ Bank | VIT